Natural Language to SQL Compiler

Compiler for Controlled English Queries

Compiler Design



Department of Computer Science and Engineering

IIITDM Kancheepuram

Developed by: Team CodeTranslators

P. Srikala (Roll No. CS23B2049)

B. Nithin (Roll No. CS23B2050)

N. Tejesh (Roll No. CS23B2051)

N. Durga Prasad (Roll No. CS23B2052)

Under the Guidance of: Dr. Maithilee Patawar

Contents

1 Abstract **3**

2 Introduction **3**

3 Importance **3**

4 Objective **4**

5 Scope of the Project **4**

6 Challenges in Natural Language to SQL Compilation **5**

7 Advantages **5**

8 System Overview **6**

9 Backend Architecture Explanation **6**

9.1 Lexical Analysis (lexer.py) 6

9.2 Syntax Analysis (parser.py) 7

9.3 Semantic Analysis (semantic_analyzer.py) 8

9.4 Code Generation (sql_generator.py) 8

9.5 Code Execution (db_executor.py) 9

10 User Manual **9**

11 Limitations **10**

12 Future Improvements **11**

13 Conclusion **11**

1 Abstract

The **Natural Language to SQL Compiler (NS2SQL)** is a compiler-inspired system that translates English queries into executable SQL statements. This project applies all major phases of compiler design — lexical, syntax, semantic analysis, code generation, and execution — using Python. It integrates compiler theory with database management concepts to help users interact with databases without manually writing SQL queries.

2 Introduction

In traditional compilers, programming languages are translated into machine code. In this project, natural language (English) is translated into SQL — which makes the system behave like a **domain-specific compiler**.

Each compiler phase corresponds to a Python module in our backend:

- **lexer.py** — Lexical Analysis
- **parser.py** — Syntax Analysis (AST Creation)
- **semantic_analyzer.py** — Semantic Validation
- **sql_generator.py** — Code Generation (SQL)
- **db_executor.py** — Code Execution (SQLite)

3 Importance

Natural Language to SQL (NL2SQL) systems are crucial in bridging the gap between non-technical users and complex database systems. Many professionals need data insights but lack SQL knowledge. This compiler enables users to retrieve information by simply typing English queries, reducing dependence on developers or analysts. It demonstrates how compiler design principles can make database systems more accessible and intelligent.

Key points of importance:

- Simplifies data access for users without SQL expertise.
- Demonstrates practical application of compiler design principles in NLP.
- Enhances human-computer interaction for database management.
- Reduces manual query formulation errors.

4 Objective

The main objectives of the **Natural Language to SQL Compiler** project are:

- To design and develop a compiler-based system that translates English sentences into SQL queries.
- To integrate compiler phases — lexical, syntax, semantic, and code generation — within the context of natural language processing.
- To provide a user-friendly interface for query translation and execution.
- To promote understanding of how compiler architecture can extend beyond programming languages.
- To demonstrate end-to-end database interaction using natural language commands.

5 Scope of the Project

The scope of this project extends to developing a functional compiler that translates natural language inputs into valid SQL queries and executes them over an existing database. It focuses on controlled English sentences that follow a predefined structure, ensuring both accuracy and interpretability.

Key aspects of the project's scope include:

- Implementation of all major compiler phases — lexical, syntax, semantic, and code generation — in Python.
- Execution of SQL statements using SQLite for demonstration and testing.
- Support for essential SQL operations like `SELECT`, `WHERE`, `UPDATE`, and `DELETE`.
- Development of a simple yet interactive web-based frontend for query input and output visualization.
- Modular and extensible design for future integration with advanced NLP models or additional databases.

Out of Scope:

- Complex query generation such as nested queries and joins with multiple tables.
- Full-fledged natural language understanding (limited to controlled English).
- Real-time voice-based query input (planned for future enhancement).

6 Challenges in Natural Language to SQL Compilation

Developing a system that accurately translates natural language into SQL involves several technical and linguistic challenges:

- **Ambiguity in Natural Language:** English sentences can have multiple interpretations, making parsing difficult.
- **Grammar Variability:** Users can phrase the same query in numerous ways — handling all patterns requires complex grammar design.
- **Mapping Words to Schema:** Identifying which words correspond to table or column names is challenging, especially with synonyms or abbreviations.
- **Semantic Validation:** Ensuring that the translated SQL query is meaningful within the existing database schema.
- **Limited Context Awareness:** Current systems lack the deep understanding required to interpret implicit intent or context.
- **Scalability:** As databases grow or schemas change, maintaining consistency between natural language interpretation and SQL generation becomes harder.

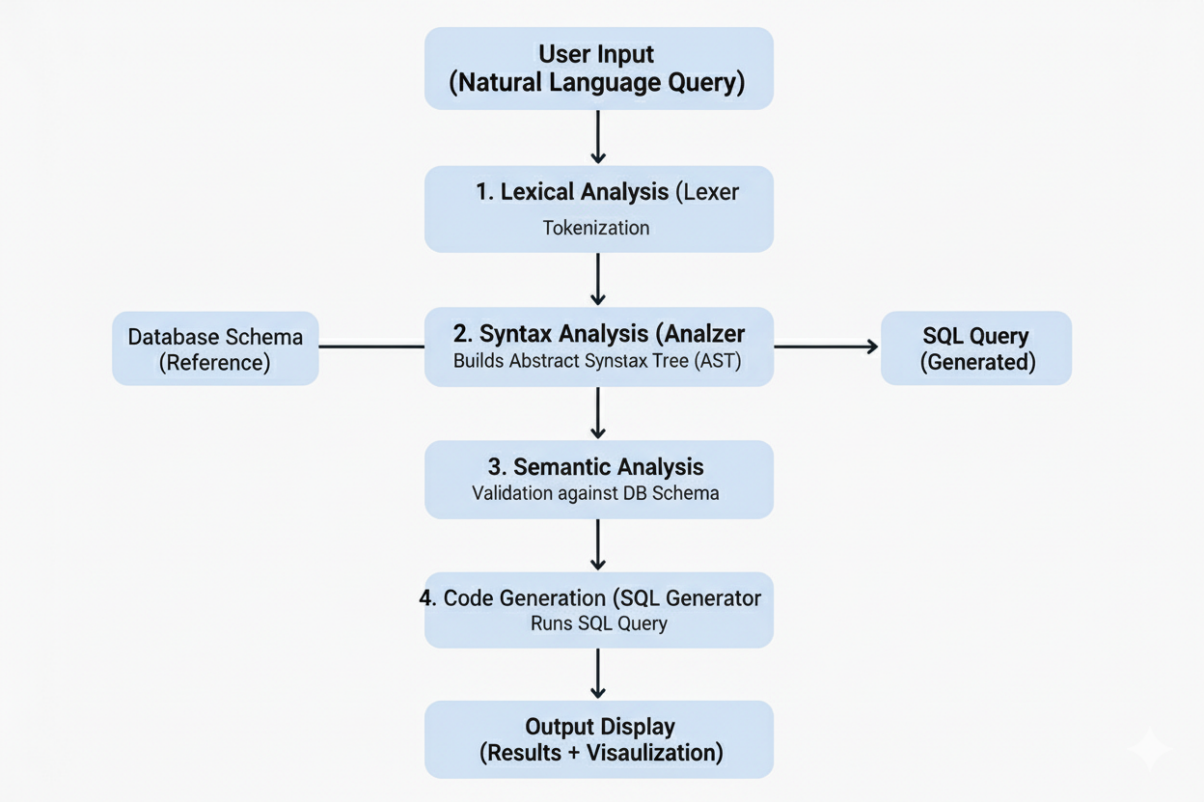
7 Advantages

The system provides a wide range of advantages both in usability and educational perspective:

- **Ease of Use:** Users can interact with databases without learning SQL.
- **Time Efficiency:** Natural language queries reduce time spent writing and debugging SQL code.
- **Practical Learning Tool:** Serves as an educational model demonstrating compiler phases in a real-world application.
- **Error Reduction:** Automates query formation, reducing human syntax or logical errors.
- **Scalability:** The modular architecture allows expansion to handle multiple database systems or NLP models.
- **Integration Capability:** Can be embedded in business intelligence tools, dashboards, and AI assistants for smart querying.

8 System Overview

The Code workflow follows classical compiler construction, with modules executing sequentially.



Wrokflow of the Code

9 Backend Architecture Explanation

9.1 Lexical Analysis (lexer.py)

Concept: Lexical analysis is the first phase of a compiler that scans the source input (English query) and breaks it into tokens. In this system, the lexer identifies SQL keywords, operators, identifiers, numbers, and strings using regular expressions.

Role in Code: It converts queries like:

```
Show employees where salary > 50000
```

into tokens such as:

```
[('SELECT', 'show'), ('IDENTIFIER', 'employees'), ('WHERE', 'where'),
 ('IDENTIFIER', 'salary'), ('GREATER', '>'), ('NUMBER', '50000')]
```

Main Code Block:

Listing 1: Core Lexical Analysis Logic

```
token_specs = [
    ('SELECT', r'\bshow\b|\bselect\b'),
    ('WHERE',  r'\bwhere\b'),
    ('GREATER', r'>'),
    ('NUMBER', r'\b\d+(\.\d+)?\b'),
    ('IDENTIFIER', r'\b[a-zA-Z_][a-zA-Z0-9_]*\b')
]

tok_regex = '|'.join(f'(?P<{t}>{r})' for t, r in token_specs)
for mo in re.finditer(tok_regex, code, re.IGNORECASE):
    yield (mo.lastgroup, mo.group().lower())
```

—

9.2 Syntax Analysis (parser.py)

Concept: The parser takes tokens and organizes them according to grammar rules to form an **Abstract Syntax Tree (AST)**. This represents the logical structure of the query.

Role in Code: It checks query patterns (e.g., SELECT → FROM → WHERE) and forms a tree of operations.

Example:

```
SelectNode(select=['name', 'salary'], table='employees', where='salary > 50000')
```

Main Code Block:

Listing 2: Core Parsing Logic

```
def parse(self):
    if self._match("SELECT"):
        return self._parse_select()

def _parse_select(self):
    self._consume("SELECT")
```

```
    columns = self._consume("IDENTIFIER").value
    self._consume("FROM")
    table = self._consume("IDENTIFIER").value
    return SelectNode(columns, table)
```

—

9.3 Semantic Analysis (semantic_analyzer.py)

Concept: Semantic analysis ensures logical correctness — verifying that tables and columns exist in the database schema.

Role in Code: Checks whether the query refers to valid tables or attributes, preventing runtime errors.

Main Code Block:

Listing 3: Semantic Validation Example

```
def analyze(self, ast):
    if ast.table not in self.schema:
        raise ValueError(f"Unknown table: {ast.table}")
    for col in ast.select_list:
        if col not in self.schema[ast.table]['columns']:
            raise ValueError(f"Unknown column: {col}")
```

—

9.4 Code Generation (sql_generator.py)

Concept: This stage acts like the compiler's code generation phase — converting the AST into SQL code.

Role in Code: It translates logical trees into valid SQL strings.

Main Code Block:

Listing 4: SQL Generation Logic

```
def generate(self, node):
    columns = ', '.join(node.select_list)
    sql = f"SELECT {columns} FROM {node.table}"
    if node.where_clause:
        sql += f" WHERE {self._visit_where(node.where_clause)}"
    return sql + ";"
```

—

9.5 Code Execution (db_executor.py)

Concept: This is the final backend phase. It executes the generated SQL query on the SQLite database using a context-managed connection.

Role in Code: Executes SQL safely and returns results as JSON for the frontend.

Main Code Block:

Listing 5: Execution Block

```
with sqlite3.connect(self.db_path) as conn:
    cursor = conn.cursor()
    cursor.execute(sql_query)
    rows = cursor.fetchall()
    return [dict(row) for row in rows]
```

—

10 User Manual

Step 1: Setup the Project

- Install Python 3.10 or above.
- Create a virtual environment:

```
python -m venv venv
venv\Scripts\activate   (Windows)
source venv/bin/activate (Linux/Mac)
```

- Install dependencies:

```
pip install fastapi uvicorn sqlite3
```

Step 2: Database Setup

- Create a folder ‘database/‘ and place your SQLite file (e.g., employees.db) inside.

Step 3: Run the Backend Server

- Create a folder ‘backend/‘ and place your backend files (e.g., lexer.py) inside.
- Open the terminal in that folder and run the below command:

```
uvicorn backend.app:app --reload
```

This starts the FastAPI server at: `http://127.0.0.1:8000`

Step 4: Access the Frontend

- Create a folder 'Frontend/' and place your Frontend Files (e.g., index.html) inside.
- Open the terminal in that folder and run the below command:

```
python -m http.server 8080
```

- Your Website is Live at: `http://localhost:8080/`
- Enter a natural query, like:

```
Show all employees where department is 'HR'
```

- Click "Compile Query" to see:
  - Lexical Tokens
  - Parse Tree
  - Generated SQL
  - Execution Result

Step 5: View Execution Result The output table will display query results fetched from your SQLite database.

—

11 Limitations

Although the system demonstrates strong accuracy for controlled English queries, a few limitations exist:

- The compiler supports only well-structured and simple English grammar; it cannot handle complex natural language sentences.
- It is limited to basic SQL constructs (SELECT, UPDATE, DELETE, CREATE).
- Nested queries and subqueries are not yet implemented.

- Semantic checking relies on a preloaded schema — new schema changes require a restart.
- The interface does not yet support multi-table aggregation in one query.

—

12 Future Improvements

There are several enhancements planned for future versions:

- Integration of NLP models such as **spaCy** or **BERT** for broader language understanding.
- Addition of query optimization techniques during SQL generation.
- Support for subqueries, joins with multiple conditions, and view creation.
- Expanding backend to work with multiple databases (MySQL, PostgreSQL).
- Improved GUI with result visualization and query explanation.
- Voice-based natural query input for accessibility.

—

13 Conclusion

The project successfully demonstrates how compiler design principles can be applied to natural language processing and database systems. Each backend module corresponds to a classical compiler phase, ensuring systematic translation, validation, and execution of queries.

—

References

1. Aho, Alfred V., Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools (2nd Edition)*, Pearson Education, 2007.
2. Fischer, C. N., and R. J. LeBlanc. *Crafting a Compiler*, Addison-Wesley, 1988.